job_ready API Documentation v0.5

Nov 8, 2026 · @DZN

Hack the Change 2026 · Front end: Sobem, Ini · Back end: Bruno, Paul · Database: Delina. This version matches the back end with real Gemini AI on every AI endpoint, the new tailor path and PDF download.

What changed since v0.4

Area	Change
Events	New `GET /events` and `GET /events/{id}`: career fairs, workshops, networking nights and info sessions, soonest first. Only events that have not ended are listed.
Schedule	**Removed.** `/schedule` (GET, POST, PATCH, DELETE) is gone and returns 404. The design has no schedule page.
Mentors	`POST /mentors` and `GET /mentors` have two new optional fields: `photoUrl` and `yearsExperience`.
Coffee chats	Every coffee chat response now includes `studentName` and `mentorName`.
Database	New table `events`, two new `mentors` columns, `schedule_items` dropped. Existing databases run `db/migrations/2026-11-08_events_mentor_fields.sql` once.

What changed since v0.3

Area	Change
AI	Every AI feature now calls a real model (Google Gemini, free tier) when the back end runs with `app.ai.provider=gemini`. AI endpoints take about 1-3 seconds, up to about 20 seconds when the free tier is busy (the back end retries and switches to a backup model by itself).
Tailor	The path changed from `POST /resumes/{id}/tailor` to **`POST /resumes/tailor`** (no resume id). The AI writes the resume from the profile plus the job. The response now has `pdfPath`.
PDF	New **`GET /resumes/{id}/pdf`** returns the resume as a PDF file (not wrapped in `Result`).
Base resume	Each user has **one** base resume. Calling `POST /resumes/generate` again overwrites it (same id).
Interviews	`POST /interviews` returns 5 questions (`iq1` to `iq5`). Answers are scored 1-10.
Errors	An invalid query value (for example `scope=moon`) now returns 400 `invalid query parameter` instead of 500. AI failures return 502 with a reason.
Local setup	H2 is gone; the back end runs on MySQL only. Test accounts and data are listed at the end.

Conventions

- Base URL: `/api/v1`. Local dev: `http://localhost:8000/api/v1`.
- All requests and responses are JSON (`Content-Type: application/json`), except `GET /resumes/{id}/pdf`.
- Every endpoint except register and login needs `Authorization: Bearer <token>`. The token comes from register or login and lasts 7 days. It is not stored on the server, so logging out means the front end deletes it.

- IDs are strings with a prefix: `u_` user, `p_` profile, `o_` opportunity, `e_` event, `r_` resume, `m_` mentor, `c_` coffee chat, `i_` interview.
- Dates use ISO 8601: `2026-11-20` or `2026-11-20T18:00:00Z`. Fields with no value are returned as `null`.
- AI output is in English.
- Every response body is a `Result`:

```
{ "code": 1, "msg": null, "data": { "...": "..." } }
```

- `code` is `1` on success and `0` on failure. `msg` carries the error message on failure. `data` holds the result (`null` for endpoints that return nothing).
- List endpoints return `data: { "items": [...], "total": n }`.
- **The request and response examples below show only the `data` part.**

Errors set an HTTP status code and return `code: 0` with a message:

```
{ "code": 0, "msg": "profile not found", "data": null }
```

Status	When
200	Success (every endpoint)
400	Bad input: missing or invalid field, malformed JSON (`malformed request`), invalid query value (`invalid query parameter`)
401	Missing or invalid token, or wrong email/password
403	Not allowed (for example a student calling `POST /mentors`)
404	Not found, or it belongs to another user. Also returned when the user has no profile yet for endpoints that need one
405	Wrong HTTP method
409	Duplicate email
502	The AI service failed (busy, rate limited, timed out). The message says why; trying again usually works

Status	When
500	Unexpected server error

Endpoint overview

Method	Path	Purpose	AI
POST	`/auth/register`	Create account (student or mentor), get token	
POST	`/auth/login`	Log in, get token	
GET	`/users/me`	Current user	
GET	`/profile/questions`	Profile questionnaire	
POST	`/profile`	Submit answers, AI builds career profile	Yes
GET	`/profile`	Get my career profile	
PATCH	`/profile`	Edit my career profile	
GET	`/opportunities`	Browse and filter opportunities	
GET	`/opportunities/matches`	Opportunities ranked by AI fit score	Yes
GET	`/opportunities/{id}`	Opportunity details	
GET	`/events`	Upcoming events, soonest first	
GET	`/events/{id}`	Event details	
POST	`/resumes/generate`	AI writes (or rewrites) my base resume	Yes
POST	`/resumes/tailor`	AI writes a new resume for one job, saves a PDF	Yes
GET	`/resumes`	List my resumes	
GET	`/resumes/{id}`	One resume	
GET	`/resumes/{id}/pdf`	Download a resume as PDF	
POST	`/resumes/{id}/suggestions`	AI suggests improvements	Yes

Method	Path	Purpose	AI
POST	`/mentors`	Sign up as mentor (or update my mentor card)	
GET	`/mentors`	Browse mentors	
POST	`/coffee-chats/icebreaker`	AI drafts a first message to a mentor	Yes
POST	`/coffee-chats`	Request a coffee chat	
GET	`/coffee-chats`	My coffee chats	
PATCH	`/coffee-chats/{id}`	Accept / decline / complete	
POST	`/interviews`	Start mock interview, AI writes questions (stretch)	Yes
POST	`/interviews/{id}/answers`	Answer, AI scores it (stretch)	Yes

Auth

POST /auth/register

Create an account. `name` is required, `email` must be valid, `password` needs at least 8 characters, `role` is `student` or `mentor`. Emails are stored in lower case. Returns 409 if the email is taken.

```
{ "name": "Paul", "email": "paul@ucalgary.ca", "password": "********",
"role": "student" }
```

Response `data` (the user is logged in right away):

```
{
  "user": { "id": "u_123", "name": "Paul", "email": "paul@ucalgary.ca",
"role": "student" },
  "token": "eyJhbGciOi..."
}
```

POST /auth/login

Log in with email (not case-sensitive) and password. Same response shape as register. A wrong email or password returns 401 `wrong email or password`.

```
{ "email": "paul@ucalgary.ca", "password": "********" }
```

GET /users/me

The logged-in user.

```
{ "id": "u_123", "name": "Paul", "email": "paul@ucalgary.ca", "role":
"student" }
```

Career profile

A profile carries education, projects and experience, using the same item shapes as resume sections.

```
{
  "id": "p_456",
  "userId": "u_123",
  "summary": "Second-year Computer Science student ...",
  "skills": ["Python", "JavaScript", "SQL"],
  "education": [{ "school": "University of Calgary", "degree": "BSc Computer
Science", "end": "2028" }],
  "projects": [{ "name": "Discord Laundry Bot", "bullets": ["Built a bot
...", "Used by 60 students"] }],
  "interests": ["Software", "Energy"],
  "experience": [{ "title": "IT Assistant", "org": "UCalgary", "start":
"2025-05", "end": "2025-08", "bullets": ["..."] }],
  "industries": ["Software", "Energy"],
  "preferredWorkType": "Startup",
  "experienceLevel": "student",
  "location": "Calgary, AB",
  "updatedAt": "2026-11-08T02:18:19Z"
}
```

- `experienceLevel`: `beginner` | `student` | `intermediate` | `senior`
- `industries`: at most 3

GET /profile/questions

Five placeholder questions so the front end has something to render. The final list is still to be decided by the team.

id	Question	Type / options
q1	Which skills are you most confident in?	text
q2	What kind of work environment do you prefer?	single_select: Startup, Large company, Research lab
q3	What year of study are you in, or how many years have you worked?	text
q4	Describe a project you are proud of.	text
q5	Where are you willing to work?	single_select: Local, Remote, Abroad, Any


```
{
  "items": [
    { "id": "q1", "text": "Which skills are you most confident in?", "type":
"text" },
    { "id": "q2", "text": "What kind of work environment do you prefer?",
"type": "single_select",
      "options": ["Startup", "Large company", "Research lab"] },
    "..."
  ],
  "total": 5
}
```


`options` is left out for text questions.

POST /profile (AI)

Submit questionnaire answers plus up to 3 industries (more than 3 returns 400). The AI reads each question with its answer and builds the profile: summary, skills, education, projects, experience, interests, preferred work type and experience level. It only uses what the answers say and leaves a field empty rather than inventing it. The answers themselves are not saved. Calling again replaces the profile (same id). Returns the profile.

```
{
  "answers": [
    { "questionId": "q1", "answer": "Python, JavaScript, a bit of SQL" },
    { "questionId": "q3", "answer": "Second year of Computer Science at
University of Calgary, graduating 2028" },
    { "questionId": "q4", "answer": "I built a Discord bot that tracks dorm
laundry machines; 60 students use it" }
  ],
  "industries": ["Software", "Energy"],
  "location": "Calgary, AB"
}
```

`answers` is required but may be an empty list.

GET /profile

The current user's profile (shape above). 404 `profile not found` if the user has not built one yet.

PATCH /profile

Edit fields by hand; send only the fields that change. Editable: `summary`, `skills`, `education`, `projects`, `interests`, `experience`, `industries`, `preferredWorkType`, `experienceLevel`, `location`. Returns the updated profile. 404 if there is no profile yet.

```
{
  "skills": ["Python", "PyTorch", "SQL"],
  "experienceLevel": "beginner"
}
```

Opportunities

One opportunity:

```
{
  "id": "o_789",
  "title": "Software Developer Co-op",
  "organization": "Example Energy Inc.",
  "type": "internship",
  "industry": "Energy",
  "location": "Calgary, AB",
  "scope": "local",
  "description": "...",
  "requirements": ["Python", "SQL"],
  "deadline": "2026-11-30",
  "url": "https://..."
}
```

- `type`: `job` | `internship` | `research`
- `scope`: `local` | `remote` | `abroad`

GET /opportunities

Browse and filter, sorted by deadline. `total` is the count across all pages.

Query parameter	Meaning
`type`	`job` | `internship` | `research` (other values return 400)
`scope`	`local` | `remote` | `abroad` (other values return 400)
`industry`	e.g. `Energy` (not case-sensitive)
`q`	Keyword in title, organization or description
`page`, `limit`	Page starts at 1. Default limit 20, max 100

GET /opportunities/matches (AI)

The AI scores every opportunity (optionally only one `scope`) against the user's profile from 0 to 1 and gives 1-3 reasons written to the user. Sorted by score, highest first. 404 if the user has no profile yet. This is the slowest AI call because all opportunities are sent at once.

Query parameter	Meaning
`scope`	Optional filter, same values as above
`limit`	Default 10

```
{
  "items": [
    { "opportunity": { "id": "o_t_001", "title": "Backend Developer Intern",
"...": "..." },
      "matchScore": 0.95,
      "reasons": ["Matches your Java, Spring Boot and MySQL skills", "Fits
your interest in backend development"] }
  ],
  "total": 1
}
```

GET /opportunities/{id}

Full details of one opportunity. 404 if it does not exist.

Events

Career fairs, workshops, networking nights and info sessions, shown on the Events side of the Opportunities page. Events are read-only for now (the team adds them to the `events` table). `kind` is `career_fair` | `workshop` | `networking` | `info_session`; `format` is `in_person` | `online`. Times are UTC; `endsAt` may be `null`.

GET /events

Upcoming events, **soonest first**. Events that have already ended are left out. Query parameters, all optional: `kind`, `format` (any other value returns 400 `invalid query parameter`), `q` (keyword in title, host, location or description), `page` (default 1), `limit` (default 20, max 100). `total` counts all matching events.

```
{
  "items": [
    {
      "id": "e_t_03",
      "title": "Newcomers in Energy: Networking Night",
      "host": "Alberta Energy Connect (fictional)",
      "kind": "networking",
      "format": "online",
      "location": "Online",
      "description": "Small-group conversations with engineers who started
their careers in Canada.",
      "startsAt": "2026-11-20T01:00:00Z",
      "endsAt": "2026-11-20T03:00:00Z",
      "url": "https://example.com/test-events/03"
    }
  ],
  "total": 4
}
```


GET /events/{id}

One event, same shape as an item above. 404 `event not found` if the id does not exist.

Resumes

A resume is stored as structured sections so the front end can render it and the AI can write it.

```
{
  "id": "r_1",
  "title": "Base resume",
  "opportunityId": null,
  "sections": {
    "summary": "...",
    "skills": ["Java", "Spring Boot", "MySQL"],
    "education": [{ "school": "University of Calgary", "degree": "BSc
Computer Science", "end": "2027" }],
    "experience": [{ "title": "...", "org": "...", "start": "2025-05", "end":
"2025-08", "bullets": ["..."] }],
    "projects": [{ "name": "...", "bullets": ["..."] }]
  },
  "createdAt": "2026-11-08T02:18:30Z"
}
```

- Base resume: `title` is `Base resume` and `opportunityId` is `null`. One per user.
- Tailored resume: `title` is `Tailored: <job title>` (or `Tailored: pasted job description`), and `opportunityId` is set when it was tailored to an opportunity.

POST /resumes/generate (AI)

The AI writes a general one-page resume from the career profile, using only facts from the profile. Body can be empty. Returns the resume.

- The first call creates the base resume. Later calls **rewrite the same resume** (same `id`, new content and `createdAt`), so a user never has more than one base resume.
- No PDF is made here; use `GET /resumes/{id}/pdf`.
- 404 if the user has no profile.

POST /resumes/tailor (AI)

The AI writes a **new** resume for one job from the user's profile. Send exactly one of `opportunityId` or `jobDescription`; both or neither returns 400. The AI may reword, reorder and select, but never invents employers, degrees, dates or skills; skills the job asks for that the user lacks are listed in `changes` instead. The PDF is saved on the server at the same time. 404 if there is no profile or the opportunity does not exist.

```
{ "opportunityId": "o_t_001" }
```

or `{ "jobDescription": "We are looking for a Python developer ..." }`. Response `data`:

```
{
  "resume": { "id": "r_2", "title": "Tailored: Backend Developer Intern",
"opportunityId": "o_t_001", "sections": { "...": "..." } },
  "changes": [
    "Highlighted SQL and analytical project experience to match the role",
    "The candidate lacks Java and Spring Boot, which the job asks for"
  ],
  "pdfPath": "Z:\\job-ready-backend\\data\\resumes\\r_2.pdf"
}
```

`pdfPath` is where the file was saved on the server (useful for debugging only). The front end should download the PDF with `GET /resumes/{id}/pdf`.

GET /resumes and GET /resumes/{id}

List the user's resumes (newest first), or get one. A resume that belongs to another user returns 404.

GET /resumes/{id}/pdf

The resume as a PDF file (`Content-Type: application/pdf`, shown inline, file name `resume-<id>.pdf`). **This response is not wrapped in `Result`.** If the PDF has not been made yet (for example a base resume), it is drawn on the first request. Errors (404) still return the JSON `Result`.

Front end: use the URL with the token header, for example `fetch(url, { headers: { Authorization: "Bearer " + token } })` and then `URL.createObjectURL(await res.blob())`. A plain `<a href>` will not send the token.

POST /resumes/{id}/suggestions (AI)

The AI gives 3-5 concrete suggestions to improve the resume. `opportunityId` is optional; with it, the suggestions focus on closing the gap to that job. The body can be empty.

```
{ "opportunityId": "o_t_002" }
```

```
{
  "suggestions": [
    { "type": "skill", "text": "Add Apache Airflow and learn the basics of
DAGs", "reason": "Airflow is a main requirement for this role" },
    { "type": "experience", "text": "Build an ETL pipeline project with
Python and SQL", "reason": "The role needs data pipeline experience" },
    { "type": "wording", "text": "Rewrite the booking bot bullet to stress
data handling", "reason": "Brings your Python work closer to data tasks" }
  ]
}
```

`type`: `experience` (something to do or build) | `wording` (how to rewrite) | `skill` (a skill to learn or show).

Mentors and coffee chats

POST /mentors

A professional signs up as a mentor. The account role must be `mentor` (otherwise 403). Calling again updates the mentor card; **fields left out are cleared**, so send the whole card. `headline` is required; `maxChatsPerWeek` defaults to 2. Returns the mentor.

```
{
  "headline": "Data Engineer",
  "company": "Example Corp",
  "industries": ["Software", "Energy"],
  "expertise": ["Data pipelines", "Career switching"],
  "availability": [{ "day": "Tue", "start": "18:00", "end": "19:00" }],
  "maxChatsPerWeek": 2,
  "photoUrl": "https://example.com/alex.jpg",
  "yearsExperience": 7
}
```

`photoUrl` (a link to an image, up to 512 characters) and `yearsExperience` (0 to 70) are optional and may be `null`; a negative or larger number returns 400. Both are also returned on every mentor card in `GET /mentors`.

GET /mentors

Browse mentors. Each item is the body above plus `id`, `userId`, `name` and `points`. Query parameters: `industry` (filter, not case-sensitive), `q` (keyword in name, headline, company or expertise).

POST /coffee-chats/icebreaker (AI)

The AI drafts a short (60-100 words), personal first message from the student to a mentor, mentioning something specific about each of them and asking for a 20-minute chat. Nothing is saved or sent: show it to the student to edit, then send it with `POST /coffee-chats`. 404 if the mentor does not exist.

```
{ "mentorId": "m_t_03" }
```

```
{ "message": "Hi Daniel, I'm Maya, a third-year Computer Science student
focused on backend development. I saw your path from Reservoir Engineer to
Data Engineer at Foothills Energy ... Would you be open to a quick 20-minute
coffee chat? Best, Maya" }
```

POST /coffee-chats

Student requests a chat. Returns the chat with status `pending`. 400 if you request a chat with yourself.

```
{
  "mentorId": "m_42",
  "message": "Hi Alex, ...",
  "proposedTimes": ["2026-11-12T18:00:00-07:00", "2026-11-14T18:00:00-07:00"]
}
```

GET /coffee-chats

Chats for the current user, as student or as mentor. `mentorId` is the mentor's id (`m_...`), not their user id.

```
{
  "items": [
    { "id": "c_9", "studentId": "u_123", "studentName": "Ini", "mentorId":
"m_42", "mentorName": "Alex Kim",
      "status": "pending", "message": "...", "proposedTimes": ["..."],
"scheduledAt": null }
  ],
  "total": 1
}
```

`studentName` and `mentorName` are in every coffee chat response: this list, `POST /coffee-chats` and `PATCH /coffee-chats/{id}`.

PATCH /coffee-chats/{id}

Only the mentor can accept or decline (otherwise 403), and only while the chat is `pending` (otherwise 400). Either side can mark an `accepted` chat `completed`, which adds 10 points to the mentor. `scheduledAt` is returned in UTC.

```
{ "status": "accepted", "scheduledAt": "2026-11-12T18:00:00-07:00" }
```

`status`: `accepted` | `declined` | `completed`.

Mock interview (stretch goal)

POST /interviews (AI)

Start a practice interview for a role or an opportunity. Send exactly one of `opportunityId` or `role`. The AI writes 5 questions: 1 opener, 2 behavioural and 2 role-specific (based on the job's requirements when an opportunity is given).

```
{ "opportunityId": "o_t_001" }
```

or `{ "role": "Data Analyst" }`. Response `data`:

```
{
  "id": "i_1",
  "questions": [
    { "id": "iq1", "text": "Could you tell me a little about yourself and why
you're interested in this internship?" },
    { "id": "iq4", "text": "We use Spring Boot; can you explain the
difference between a Controller, a Service and a Repository?" },
    "..."
  ]
}
```

POST /interviews/{id}/answers (AI)

Submit one answer and get a score from 1 to 10 with feedback written to the user. Each answer and its feedback are saved on the interview; answering the same question again replaces the earlier answer. `nextQuestion` is `null` after the last question. 404 if the question id does not exist.

```
{ "questionId": "iq2", "answer": "At my IT support job I noticed laptop setup
took 30 minutes ..." }
```

```
{
  "score": 7,
  "feedback": "Great job showing initiative and a measurable result. To
improve, ...",
  "nextQuestion": { "id": "iq3", "text": "..." }
}
```

Database tables (for Delina)

MySQL database `job_ready_db`. The full script is `db/schema.sql` in the back-end repo. v0.5 added `events` and two `mentors` columns, and dropped `schedule_items`. A database made from an older script needs `db/migrations/2026-11-08_events_mentor_fields.sql` run once.

Table	Columns
users	id, name, email, password_hash, role (student | mentor), created_at

Table	Columns
profiles	id, user_id (unique), summary, skills, education, projects, interests, experience, industries, preferred_work_type, experience_level (beginner | student | intermediate | senior), location, updated_at
opportunities	id, title, organization, type, industry, location, scope, description, requirements, deadline, url, created_at
events	id, title, host, kind, format (in_person | online), location, description, starts_at, ends_at (DATETIME, UTC), url, created_at
resumes	id, user_id, opportunity_id, title, sections (JSON), created_at
mentors	id, user_id, headline, company, industries, expertise, availability (JSON), max_chats_per_week (default 2), points, photo_url, years_experience
coffee_chats	id, student_id, mentor_id (→ mentors.id), status, message, proposed_times, scheduled_at (DATETIME, UTC), created_at
interviews	id, user_id, opportunity_id, role, questions (JSON), answers (JSON), created_at

List and object columns (skills, requirements, sections, availability, education, …) are MySQL `JSON`. All ids are `VARCHAR(64)`.

Running the back end locally

1. Create the database once: `mysql -uroot -p < db/schema.sql`.
2. Copy `src/main/resources/application-dev.yml.example` to `application-dev.yml` (git-ignored) and fill in your MySQL password and a free Gemini API key from https://aistudio.google.com/apikey.
3. Start: `./mvnw spring-boot:run` (Windows: `.\mvnw.cmd spring-boot:run`), or run `JobReadyApplication` in IntelliJ. Port 8000.

Useful local settings in `application-dev.yml`:

Setting	Effect
`app.ai.provider: gemini`	Real AI. `stub` gives offline rule-based results (used by the automated tests)
`app.ai.model` / `fallback-model`	Default `gemini-3.5-flash-lite`, backup `gemini-3.1-flash-lite`
`app.auth.mode: demo` + `demo-user-email`	Requests with no token or an invalid token act as that user. Never use outside local testing
`app.test-data.enabled: true` + `sql-file`	Loads `db/test-data.sql` on startup when the newest test user is missing
`app.resume.pdf-dir`	Folder where resume PDFs are saved (default `data/resumes`)

Test data (`db/test-data.sql`, all fictional, password `password123` for every test account):

Account	Role	Notes
maya.chen@test.jobready.dev	student	Backend / Java, full profile
liam.patel@test.jobready.dev	student	Data / statistics
sofia.rossi@test.jobready.dev	student	Mechanical engineering / renewables
noah.kim@test.jobready.dev	student	Front end / UX, beginner
aisha.bello@test.jobready.dev	student	Machine learning for health
ethan.wong@test.jobready.dev	student	**No profile**, for testing the questionnaire
olivia.martin@test.jobready.dev	student	Business analysis
lucas.nguyen@test.jobready.dev	student	Embedded / IoT
alex.morgan@test.jobready.dev, priya.shah@..., daniel.lee@..., grace.okafor@...	mentor	Mentor cards `m_t_01` to `m_t_04`

There are also 20 test opportunities (`o_t_001` to `o_t_020`), 11 test events (`e_t_00` , already over, to `e_t_10`) and coffee chats in every state.

Code layout (like 苍穹外卖): `controller` → `service` / `service/impl` → `mapper` (MyBatis, SQL in `resources/mapper/*.xml`); `pojo/entity` , `pojo/dto` (request bodies), `pojo/vo` (response shapes); `ai/AiClient` with `GeminiAiClient` (real) and `StubAiClient` (offline); `interceptor/JwtTokenInterceptor` puts the user id in `BaseContext` ; `handler/GlobalExceptionHandler` turns errors into `Result` .